

Studio Onboarding (Init Wizard)

Last updated: 2026-05-11

The init wizard is the gated setup flow that connects a studio's source tool (Airtable or Jira) to ArtHound and runs the first full sync. It must be completed before any ArtHound product features are accessible. The wizard walks through credentials, source discovery, P→A→W hierarchy definition, field classification, and finally kicks off the initial replication job.

Prerequisites

- The studio org exists and the user has at least `admin` role
 - For Jira Cloud: OAuth 2.0 credentials must already be obtained via the Jira OAuth flow before the wizard begins
 - For Airtable: a Personal Access Token (PAT) and Base ID are required
-

Steps

The wizard enforces sequential gate-checking. Each step can only be reached if the preceding step has been completed.

Step 1: Credentials (`POST /api/init/credentials`)

Validates and saves the source credentials encrypted in `source_credentials`.

- **Airtable:** accepts `api_token` + `base_id`; makes a test API call and returns table/field counts as confirmation
- **Jira:** credentials are pre-stored via the OAuth callback; this step validates the token is still active

On success, the step is marked complete and the wizard advances.

Step 2: Discover (`POST /api/init/discover`)

Fetches the full base/project schema from the source tool and caches it in `source_schema_cache`. This cache powers all the table and field selectors in subsequent steps.

Returns a summary (table names, field counts). The raw schema is retrieved by the wizard via `GET /api/init/schema-cache` as needed.

Step 3: Entity Definitions (GET / PUT /api/init/entity-definitions)

Defines the studio's P→A→W hierarchy, which source tables correspond to Products, Assets, and Work, and how they link together. This is the most complex step.

For each entity (Product, Asset, Work): - **Table ID**: which source table contains this entity's records - **Filter**: optional formula/JQL to restrict which records are included (e.g., `Type = "Character"`) - **Link field**: which field on the Asset record holds the link to its Product (and which field on the Work record holds the link to its Asset) - **Link direction**: whether the parent holds the link (`parent_holds_link`: the Product record has a multi-link to Assets) or the child does (`child_holds_link`: the Asset record has a single-select pointing to its Product)

Work-specific fields are also captured here: - `task_name_field_id` / `task_name_field_name` - `task_status_field_id` / `task_status_field_name` - `task_start_date_field_id`, `task_end_date_field_id` - `task_estimate_field_id`

These are persisted in `source_entity_definitions`. The sync layer reads exclusively from this table, the connector never assumes source structure.

`POST /api/init/preview-entity` validates a definition against live source data and returns sample records, letting the studio confirm their filters return what they expect before committing.

Step 4: Field Mappings (GET / PUT /api/init/field-mappings)

Maps source fields to ArtHound's named asset slots and classifies all fields by meta bucket and display tier.

Required slots that must be covered before the wizard can advance: `name`, `status`, `item_type`.

Field classification: - **Meta bucket**: `production`, `scheduling`, `technical`, `business`, `custom`, `source_native` - **Display tier**: `primary` (shown by default in Details tab), `secondary` (collapsed behind "Show more"), `hidden` (never rendered) - **Ingest suppressed**: fields with `[IGNORE]` prefix or explicitly suppressed; excluded from `meta` storage entirely

The normalizer auto-generates initial mappings by matching field names against slot aliases. The wizard lets studios review and override these before the first sync runs. See [Sync Layer, Field Normalization](#) for how slots and aliases work.

Step 5: Start (POST /api/init/start)

Gate-checks: 1. Credentials exist in `source_credentials` 2. Required slots (`name`, `status`, `item_type`) are covered in field mappings

On pass: - Creates an `init_jobs` row (`status = pending`) - Enqueues a full sync as a background task

Returns `{job_id}` for polling.

Polling (GET /api/init/jobs/{job_id})

Returns:

```

{
  "status": "pending | running | success | error",
  "phase": "fetching assets | normalizing | writing ... ",
  "progress_current": 240,
  "progress_total": 1500,
  "error_log": [...],
  "started_at": "...",
  "completed_at": "... "
}

```

The wizard polls this until `success` or `error`. On success, the studio's `initialized_at` timestamp is set and they are redirected to the main product.

Additional Routes

Method	Path	Description
POST	<code>/api/init/backfill-buckets</code>	Backfill meta bucket classification on existing mappings (idempotent; safe to run multiple times)
POST	<code>/api/init/reset</code>	Wipe all replicated data for the studio, soft-delete generated work, re-enqueue init. Prompts a warning in the UI showing how many records will be deleted.

Source Entity Definitions Schema

`source_entity_definitions`, one row per (owner, PAW level):

```

owner_type:    studio | vendor
owner_id:      uuid
paw_level:     product | asset | work | item_type
table_id:      source table identifier
filter_formula: optional source-tool filter expression
rel_field_id:  the linking field (parent or child side)
rel_direction: parent_holds_link | child_holds_link
task_name_field_id, task_status_field_id, ...

```

This table is the single source of truth for "what does this studio's source look like." The sync layer, scheduler, field mapping UI, and init wizard all read from it. Nothing about source structure is hardcoded elsewhere.

Known Gaps

No item-type source variety: item types can be fetched from a separate table or inferred from a field's select values on the asset table. The wizard captures this, but the detailed mapping UI for non-asset item-type sources is limited.

Work field mapping UI: the task field mappings (`task_name_field_id` , etc.) are captured in entity definitions, but there is no dedicated work field mapping UI equivalent to the asset field mapping screen. Work fields beyond the named slots go into `meta` without classification.

No Product field mapping: same gap as the sync layer: product fields beyond `name` have no slot mapping and no classification UI.